

Full Load vs Incremental Merge vs Incremental Append

Three ways a dbt model can load data, explained with this project's own tables: customer, account, and hist_transactional.

Meaning

	Row untouched (not re-read, not re-written)
	Row updated in place
	Brand-new row
	Row no longer present

1. Full Load — customer table

Every time the model runs, dbt **throws away the whole target table and rebuilds it from scratch** by re-reading the entire source table. There is no "what changed" logic at all — the target is always an exact mirror of the source at the moment the model ran.

dbt config: materialized = 'table'

Day 1 — source customer table

cust_id	name	email
C001	Ram	ram@test.com
C002	Sita	sita@test.com
C003	Hari	hari@test.com

Day 1 target after the first run: identical copy of the source.

Day 2 — source customer table (Sita's email changed, Hari removed, Gita added)

cust_id	name	email
C001	Ram	ram@test.com
C002	Sita	sita_new@test.com
C004	Gita	gita@test.com

Day 2 target after re-running the full load

cust_id	name	email
C001	Ram	ram@test.com
C002	Sita	sita_new@test.com
C004	Gita	gita@test.com

Hari (C003) is gone from the target too — not because anyone deleted it there, but because the **WHOLE** table got replaced with whatever the source looked like on Day 2. Full load handles updates, inserts, AND deletes automatically — but it re-reads and re-writes every single row, every single run, even the ones that never changed.

2. Incremental Merge — account table

Only rows that are **new or changed since the last run** are read from the source (detected via a "last changed" column — in this project, `lchg_time`). Those rows are then **merged** into the existing target using a unique key: if the key already exists in the target → **UPDATE** that row in place; if it doesn't exist yet → **INSERT** a new row. Every other row in the target is left completely untouched.

```
dbt config: materialized = 'incremental', incremental_strategy = 'merge', unique_key = 'account_id'
```

Day 1 — target account table (after the first run)

account_id	balance	lchg_time
AC001	1,000	Day 1
AC002	2,000	Day 1
AC003	3,000	Day 1

Day 2 — what changed in the source (AC002's balance changed, AC004 is brand new)

account_id	balance	lchg_time
AC001	1,000	Day 1
AC002	2,500	Day 2
AC003	3,000	Day 1
AC004	4,000	Day 2

The incremental filter (`WHERE lchg_time > MAX(lchg_time) FROM target`) pulls in ONLY the 2 highlighted rows — AC001 and AC003 are not even read from the source this run.

Day 2 target after the incremental merge run

account_id	balance	lchg_time
AC001	1,000	Day 1
AC002	2,500	Day 2
AC003	3,000	Day 1
AC004	4,000	Day 2

AC001 and AC003 are byte-for-byte the same rows dbt wrote on Day 1 — they were never re-processed. AC002 was updated in place (same row, new balance). AC004 is a new row, appended. This is exactly how this project's own `stg_account.sql` works.

3. Incremental Append — hist_transactional table

Only **new** rows since the last run are read from the source (detected via `modified_date`), and they are simply **added on top** of the target — there is no unique-key matching and no UPDATE step at all. Existing rows are never touched, because a transaction that already happened doesn't change — history is immutable, we only ever add new events.

```
dbt config: materialized = 'incremental', incremental_strategy = 'append'
```

Day 1 — target hist_transactional table (after the first run)

tran_id	account_id	amount	tran_date
TXN001	AC001	500	Day 1
TXN002	AC002	700	Day 1

Day 2 — two new transactions happen (nothing about Day 1's rows changes)

tran_id	account_id	amount	tran_date
TXN003	AC003	900	Day 2
TXN004	AC001	300	Day 2

Day 2 target after the incremental append run

tran_id	account_id	amount	tran_date
TXN001	AC001	500	Day 1
TXN002	AC002	700	Day 1
TXN003	AC003	900	Day 2
TXN004	AC001	300	Day 2

TXN001 and TXN002 are exactly as they were on Day 1 — append never rewrites existing rows, not even to fix a mistake. The two new rows are simply inserted on top. This is exactly how this project's own `stg_hist_transactional.sql` works.

Side-by-side comparison

	Full Load (customer)	Incremental Merge (account)	Incremental Append (hist_transactional)
Reads from source	Entire table, every run	Only new/changed rows (via lchg_time)	Only new rows (via modified_date)
Effect on target	Whole table replaced	Matched rows updated; new rows inserted; others untouched	New rows appended; existing rows never touched
Needs a unique key?	No	Yes	No
Handles source updates?	Yes (automatic)	Yes (explicit, only for changed rows)	No — history is immutable
Handles source deletes?	Yes (automatic)	No	No (not applicable)
dbt config	materialized='table'	materialized='incremental', strategy='merge'	materialized='incremental', strategy='append'
Best for	Small dimensions, no reliable change-tracking column	Slowly-changing entities (accounts, customers)	Immutable event/log data (transactions, clicks)

One-line summary: **Full load** = reload everything, every time. **Incremental merge** = update what changed, insert what's new, leave the rest alone. **Incremental append** = only ever add new rows, never touch what's already there.